

University of Tehran
College of Engineering
School of Electrical and
Computer Engineering

Machine Learning

Assignment 3

Name and Surname
Rouja Aghajani

Student Number
810101380

December 8, 2025

Contents

1 Abstract **1**

2 Question 1: Support Vector Machines (SVM) and Kernel Methods **2**

2.1 Why the Dual Formulation is Preferred When Using Kernels . 2

2.2 Why Only Support Vectors Influence the Final Classifier . . . 3

2.3 Why the Kernel Trick Works Without Explicit Feature Computation 4

2.4 Effect of Kernel Choice on Boundary Geometry and Generalization 5

2.4.1 Linear Kernel: $k(x, x') = x^\top x'$ 5

2.4.2 Polynomial Kernel: $k(x, x') = (\gamma x^\top x' + r)^d$ 5

2.4.3 RBF Kernel: $k(x, x') = \exp(-\gamma \|x - x'\|^2)$ 5

2.4.4 Summary Insight 6

3 Question 3: Symmetric 4-point SVM (hard-margin, linear kernel) **9**

3.1 Dual problem formulation 9

3.2 Use symmetry to reduce variables 10

3.3 Substitute into the dual objective and maximize 11

3.4 Recover primal parameters w and b 12

3.5 Geometric interpretation and quick checks 13

4 Question 4: Explicit cubic feature map for the kernel $K(x, z) = (1 + x^\top z)^3$ **14**

4.1 (1) Expansion and dimensionality 14

4.2 (2) Explicit feature map $\varphi(x) \in \mathbb{R}^{10}$ 15

4.3 (3) Verification: $\varphi(x)^\top \varphi(z) = (1 + x^\top z)^3$ 17

5 Question 5: The Mechanics of Splitting (ID3) 19

5.1 Root entropy $H(Y)$ 19

5.2 Information Gain: Gain(S, X_1) and Gain(S, X_2) 20

5.3 Tree construction and training error 21

5.4 Would a bottom-up approach necessarily give the same tree? . 21

6 Question 6: Optimization Geometry & Jensen's Inequality 23

6.1 Proof of concavity (compute $\frac{d^2H}{dp^2}$) 23

6.2 Jensen's Inequality $\Rightarrow$ Information Gain ≥ 0 24

6.3 Optimization trap: a concrete scenario where greedy ID3 fails 25

7 Question 7: The Failure of Greedy Search (XOR) 28

7.1 The Greedy Trap: compute Gain(S, X_1) and Gain(S, X_2) . . . 28

7.2 Depth–2 decision tree that perfectly classifies XOR 30

7.3 Inductive bias of ID3: Preference vs Restriction 30

8 Question 8: From Data to Diagnosis — Diabetes Classification Using SVM 33

8.1 Part 1: Data Preprocessing and Exploration 33

8.2 Part 2: Linear SVM Classification 34

8.3 Part 3: Comparison of Different Kernels 35

8.4 Part 4: Hyperparameter Tuning (RBF Kernel) 36

8.5 Part 5: Dimensionality Reduction and Visualization 37

8.6 Part 6: Class Imbalance Analysis 39

8.7 Summary of Findings 40

9 Question 9: QP-SVM Primal Challenge — Soft-Margin Linear SVM 41

9.1 1. Data Preprocessing 41

9.2 2. QP Formulation and Solution 42

9.3 3. Model Evaluation 43

9.4 4. Visualizing the Decision Boundary 43

9.5 5. Detailed Analysis and Interpretation 45

10 Question 10: Decision Tree Classifier Project 46

10.1 (a) Dataset Description 46

10.2 (b) Data Preprocessing and High Cardinality Trap 46

10.3 (c) Entropy and Information Gain Engine 47

10.4 (d) Handling Continuous Features 47

10.5 (e) Recursive Tree Construction 48

10.6 (f) Overfitting Analysis 48

10.7 (g) Pruning Methods 49

10.8 (h) Summary 49

List of Figures

1 Data Processed 34
2 True Label vs Predicted Label 35
3 Linear: True Label vs Predicted Label 35
4 RBF: True Label vs Predicted Label 36
5 Poly: True Label vs Predicted Label 37
6 Training Data 38
7 SVM Decision Boundary 38
8 Comparison 39
9 Preprocessing 42
10 True vs Predicted 44
11 Decision Boundary 44
12 Post Pruning 49

List of Tables

1 Comparison of different SVM kernels on test data. 36
2 Comparison of SVM kernels on the Pima Indians Diabetes Dataset. 36
3 Summary of Tree Behaviors under Clean vs. Noisy Data and Pruning 49

Abstract

This work presents a coherent exploration of both Support Vector Machines (SVMs) and Decision Trees (ID3), moving from core theoretical insights to practical implementation and analysis. Along the SVM path, we explain why the dual is preferred when using kernels, how only support vectors affect the classifier, and how the kernel trick enables computations in high (even infinite) dimensions while shaping model capacity and susceptibility to overfitting. We then examine the RBF kernel's implicit infinite feature expansion, solve a small linear SVM analytically, and construct a 10-dimensional explicit cubic polynomial feature map. The decision-tree portion develops entropy and Jensen's inequality, illustrates the pitfalls of greedy splitting (as in XOR), and distinguishes preference bias from restriction bias. The programming sections build full pipelines: SVMs with scaling, kernel comparison, and hyperparameter tuning on the Pima diabetes dataset; a primal soft-margin SVM solved as a quadratic program; and an ID3 implementation extended to continuous attributes, with experiments on noise sensitivity, depth growth, and post-pruning. Across all parts, the central lessons are understanding the geometry and capacity of models, how algorithmic decisions affect generalization, and how theoretical properties translate into empirical performance.

Question 1: Support Vector Machines (SVM) and Kernel Methods

In this question, we explore the deeper reasoning behind why the dual SVM formulation benefits kernel methods, what support vectors actually do, why non-support vectors vanish from the model, how kernels implicitly provide infinite-dimensional feature spaces, and how different kernels change the geometry and generalization behavior of SVM classifiers.

2.1 Why the Dual Formulation is Preferred When Using Kernels

The primal soft-margin SVM seeks a vector w directly in the feature space:

$$\min_{w,b,\xi} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i, \qquad \text{s.t. } y_i(w^\top\phi(x_i)+b) \geq 1-\xi_i.$$

The dual form eliminates w entirely:

$$\max_{\alpha} \quad \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i,j}\alpha_i\alpha_j y_i y_j \langle\phi(x_i),\phi(x_j)\rangle, \quad \text{s.t. } 0\leq\alpha_i\leq C,\ \sum_i \alpha_i y_i = 0.$$

Why Dual Helps with Kernels:

- **Reason 1 (only inner products appear):** In the dual, all interactions between training points occur through

$$\langle\phi(x_i),\phi(x_j)\rangle,$$

which we can replace by a kernel $k(x_i,x_j)$. We never need to explicitly

construct ϕ. This is algebraically impossible to do in the primal.

- **Reason 2 (computational feasibility):** The primal depends on the dimension of feature space (potentially infinite). The dual depends only on the $n \times n$ kernel matrix K. Thus even an infinite-dimensional mapping can be handled because the optimization never references w directly.

2.2 Why Only Support Vectors Influence the Final Classifier

The final classifier is expressed entirely using the dual variables:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i k(x_i, x) + b.$$

Thus, a training point influences f *only* if $\alpha_i > 0$.

To understand why many α_i become zero, we refer to the Karush–Kuhn–Tucker (KKT) conditions of optimality:

$$\alpha_i \Big(y_i(w^\top \phi(x_i) + b) - 1 + \xi_i \Big) = 0.$$

This produces three regimes:

- If $y_i(w^\top \phi(x_i) + b) > 1$ (strictly outside the margin):

$$\Rightarrow \alpha_i = 0.$$

  These points do not affect the boundary.

- If $y_i(w^\top \phi(x_i) + b) = 1$ (exactly on the margin):

$$0 < \alpha_i < C.$$

  Such points become *support vectors*.

- If inside or violating the margin:

$$\alpha_i = C$$

(also nonzero) thus influencing the model.

Conclusion: Only the geometry of the boundary matters, not points far from it, explaining why SVM solutions are sparse.

2.3 Why the Kernel Trick Works Without Explicit Feature Computation

In the dual optimization, both the objective and classifier depend only on inner products:

$$\langle \phi(x_i), \phi(x_j) \rangle.$$

If a kernel function satisfies

$$k(x, x') = \langle \phi(x), \phi(x') \rangle,$$

then computations involving ϕ can be replaced with evaluations of k.

Why this avoids explicit mapping:

Consider the RBF kernel:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2).$$

Its Mercer expansion shows that it corresponds to an infinite-dimensional Hilbert space:

$$k(x, x') = \sum_{m=1}^{\infty} \lambda_m \psi_m(x) \psi_m(x').$$

But the algorithm never forms the ψ_m nor stores $\phi(x)$; it merely evaluates $k(x, x')$.

Thus: Kernel evaluations allow SVMs to act as if they were operating in infinite dimensions, while remaining computationally finite.

2.4 Effect of Kernel Choice on Boundary Geometry and Generalization

Different kernels implicitly induce different feature spaces and therefore different hypothesis classes.

2.4.1 Linear Kernel: $k(x, x') = x^\top x'$

- **Decision boundary:** A hyperplane.
- **Capacity:** Low bias, interpretable; useful when data is linearly structured.
- **Overfitting risk:** Usually small; governed mostly by C.

2.4.2 Polynomial Kernel: $k(x, x') = (\gamma x^\top x' + r)^d$

- **Decision boundary:** Polynomial surface of degree d.
- **Capacity:** Grows sharply with d; nonlinear interactions modeled.
- **Overfitting risk:** High when d is large relative to data.

2.4.3 RBF Kernel: $k(x, x') = \exp(-\gamma \|x - x'\|^2)$

- **Decision boundary:** Highly flexible, curved, local.
- **Capacity:** Very high (infinite-dimensional feature space).
- **Overfitting risk:** Controlled by (C, γ); too large γ overly localizes the classifier.

2.4.4 Summary Insight

$$\text{Linear} \;<\; \text{Polynomial} \;<\; \text{RBF}$$

in boundary flexibility and capacity. The regularization constant C and hyperparameters (d, γ) determine whether this flexibility is beneficial or leads to overfitting.

Final Result: Kernelss change the geometry of classification; the dual formulation makes their use feasible; and only the geometrically critical training points (support vectors) persist in the final classifier.

Question 2: RBF kernel feature space intuition

We consider the Gaussian (RBF) kernel

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right).$$

To show that K corresponds to an inner product in an *infinite-dimensional* Hilbert space, we expand K into a series that resembles the dot product of infinitely many basis functions.

Step 1: Rewrite the squared distance.

$$\|x - z\|^2 = \|x\|^2 + \|z\|^2 - 2x^\top z$$

Thus:

$$K(x, z) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right) \exp\left(-\frac{\|z\|^2}{2\sigma^2}\right) \exp\left(\frac{x^\top z}{\sigma^2}\right)$$

Step 2: Expand the cross term using its power series.

$$\exp\left(\frac{x^\top z}{\sigma^2}\right) = \sum_{k=0}^{\infty} \frac{1}{k!} \left(\frac{x^\top z}{\sigma^2}\right)^k$$

Each term $\left(x^\top z\right)^k$ can itself be expanded in terms of all k-order monomials of x and z (multi-index notation):

$$(x^\top z)^k = \sum_{|\alpha|=k} \frac{k!}{\alpha_1!\alpha_2! \cdots \alpha_d!} x^\alpha z^\alpha$$

Hence every power k contributes a *full symmetric tensor* of monomials of degree k.

Step 3: Group the expansion into inner products of basis functions. We can define the (infinite) feature map:

$$\phi(x) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)\left(\phi_0(x), \phi_1(x), \phi_2(x), \dots\right)$$

where $\phi_k(x)$ contains *all* degree-k monomials of x scaled by σ and the factorial terms from the expansion. Formally,

$$K(x, z) = \langle \phi(x),\, \phi(z) \rangle$$

where the inner product is taken in an ℓ^2 space of infinitely many coordinates.

Result: The RBF kernel implicitly maps x into a vector consisting of every possible monomial of every degree. Since there are infinitely many degrees k, the induced feature map lives in an infinite-dimensional Hilbert space. The kernel never explicitly constructs $\phi(x)$; evaluating $K(x, z)$ alone is equivalent to computing their inner product in that space.

Thus,

$K(x, z)$ acts as an inner product in an infinite-dimensional feature space without explicitl

Question 3: Symmetric 4-point SVM (hard-margin, linear kernel)

What the question asks. We are given four points placed symmetrically at the corners of a unit square in $\mathbb{R}^2$:

$$x_1 = (1,1)^\top,\ y_1 = +1, \qquad x_2 = (1,-1)^\top,\ y_2 = +1,$$

$$x_3 = (-1,1)^\top,\ y_3 = -1, \qquad x_4 = (-1,-1)^\top,\ y_4 = -1.$$

Using a hard-margin SVM with the linear kernel (i.e. $\phi(x) = x$), solve the dual problem to obtain the optimal dual variables α_i. Then compute the primal parameters w and b. Because the data are highly symmetric, we will exploit symmetry to simplify the algebra and present the solution step by step.

3.1 Dual problem formulation

The hard-margin SVM dual is

$$\begin{aligned} \max_{\alpha \in \mathbb{R}^4} \quad & \sum_{i=1}^{4} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{4} \sum_{j=1}^{4} \alpha_i \alpha_j y_i y_j\, x_i^\top x_j, \\ \text{s.t.} \quad & \sum_{i=1}^{4} \alpha_i y_i = 0, \qquad \alpha_i \geq 0 \ \ (i = 1, \ldots, 4). \end{aligned}$$

Compute the Gram (inner product) matrix G with entries $G_{ij} = x_i^\top x_j$. Using

the given points:

$$x_1^\top x_1 = 2, \quad x_2^\top x_2 = 2, \quad x_3^\top x_3 = 2, \quad x_4^\top x_4 = 2,$$
$$x_1^\top x_2 = 0, \quad x_1^\top x_3 = 0, \quad x_1^\top x_4 = -2,$$
$$x_2^\top x_3 = -2, \quad x_2^\top x_4 = 0, \quad x_3^\top x_4 = 0.$$

Thus (ordering indices $1, 2, 3, 4$)

$$G = \begin{pmatrix} 2 & 0 & 0 & -2 \\ 0 & 2 & -2 & 0 \\ 0 & -2 & 2 & 0 \\ -2 & 0 & 0 & 2 \end{pmatrix}.$$

Define Q with $Q_{ij} = y_i y_j G_{ij}$. Since $y_1 = y_2 = +1,\ y_3 = y_4 = -1$, one finds

$$Q = \begin{pmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{pmatrix}.$$

So the dual objective is

$$L(\alpha) = \sum_{i=1}^{4} \alpha_i \ - \ \tfrac{1}{2}\, \alpha^\top Q \alpha,$$

with the linear equality constraint $\alpha_1 + \alpha_2 - \alpha_3 - \alpha_4 = 0$ and $\alpha_i \geq 0$.

3.2 Use symmetry to reduce variables

The data are symmetric under exchanging the two positive points (x_1, x_2) and under exchanging the two negative points (x_3, x_4). It is natural to seek a sym-

metric solution

$$\alpha_1 = \alpha_2 =: a, \qquad \alpha_3 = \alpha_4 =: b.$$

The equality constraint becomes

$$a + a - b - b = 0 \quad \Longrightarrow \quad 2a - 2b = 0 \quad \Longrightarrow \quad a = b.$$

Therefore all four dual variables are equal; set

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 =: \alpha \geq 0.$$

3.3 Substitute into the dual objective and maximize

Sum of α_i is 4α. The quadratic term is

$$\alpha^\top Q \alpha = \sum_{i,j} \alpha\alpha Q_{ij} = \alpha^2 \sum_{i,j} Q_{ij}.$$

Compute the sum of all entries of Q. Each row sums to 4 (check any row), so the total is 16. Thus

$$\alpha^\top Q \alpha = 16\alpha^2.$$

Hence the dual objective reduces to a one-dimensional quadratic

$$L(\alpha) = 4\alpha - \tfrac{1}{2} \cdot 16\alpha^2 = 4\alpha - 8\alpha^2.$$

Maximize over $\alpha \geq 0$: take derivative

$$\frac{dL}{d\alpha} = 4 - 16\alpha \quad \Rightarrow \quad \alpha^\star = \frac{4}{16} = \frac{1}{4}.$$

The second derivative is $-16 < 0$, so this is a maximum and $\alpha^\star > 0$ is feasible.

Optimal dual variables: $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{4}$.

3.4 Recover primal parameters w and b

The primal weight vector is

$$w = \sum_{i=1}^{4} \alpha_i y_i x_i.$$

With $\alpha_i = \frac{1}{4}$ and the given y_i, x_i:

$$\begin{aligned} w &= \tfrac{1}{4}\big(+1 \cdot x_1 + 1 \cdot x_2 - 1 \cdot x_3 - 1 \cdot x_4\big) \\ &= \tfrac{1}{4}\big((1,1) + (1,-1) + (1,-1) + (1,1)\big) \end{aligned}$$

(note the negatives cancel because $-1 \cdot (-1,1) = (1,-1)$ and $-1 \cdot (-1,-1) = (1,1)$). Summing components:

$$w = \big(1,\ 0\big)^\top.$$

Thus $\|w\| = 1$. The margin is $1/\|w\| = 1$.

To find b, use the fact that for hard-margin SVM any support vector with $0 < \alpha_i$ satisfies

$$y_i(w^\top x_i + b) = 1.$$

All four points are support vectors here (all $\alpha_i > 0$). Use $x_1 = (1,1)$ with $y_1 = +1$:

$$1 \cdot \big(w^\top x_1 + b\big) = 1 \quad \Rightarrow \quad w^\top x_1 + b = 1.$$

Compute $w^\top x_1 = (1,0) \cdot (1,1) = 1$, so $1 + b = 1$ and hence $b = 0$. This is consistent with any other support vector (e.g. for x_3, $w^\top x_3 = -1$ and $-1 + b = -1$).

$$\boxed{w = (1,0)^\top, \qquad b = 0.}$$

3.5 Geometric interpretation and quick checks

- The separating hyperplane is $w^\top x + b = x_1 = 0$, i.e. the vertical line through the origin. Positives lie at $x_1 = +1$, negatives at $x_1 = -1$.
- The margin equals 1, since $\|w\| = 1$ and margin $= 1/\|w\|$.
- All four training points are support vectors (they all lie at distance exactly 1 from the hyperplane), which is why none of the α_i vanish.
- The obtained solution satisfies the primal–dual consistency (KKT) and the equality constraint $\sum_i \alpha_i y_i = 0$ (here $4 \cdot \frac{1}{4} - 4 \cdot \frac{1}{4} = 0$).

Final Answer:

$$\boxed{\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \tfrac{1}{4}, \qquad w = (1,0)^\top, \qquad b = 0.}$$

Question 4: Explicit cubic feature map for the kernel $K(x, z) = (1 + x^\top z)^3$

What the question asks. We must (1) expand the kernel $(1 + x^\top z)^3$ using the multinomial theorem and count how many coordinates the corresponding feature map has; (2) exhibit an explicit $\varphi : \mathbb{R}^2 \to \mathbb{R}^{10}$ (list all 10 components) with the correct combinatorial *square-root* scalings so that

$$K(x, z) = \varphi(x)^\top \varphi(z);$$

and (3) verify by computing the dot product that the proposed φ reproduces the expanded polynomial. The aim is an exact, non-generic construction (not just "there exists ϕ").

4.1 (1) Expansion and dimensionality

Write $x = (x_1, x_2)^\top$, $z = (z_1, z_2)^\top$. Let

$$s := 1 + x^\top z = 1 + x_1 z_1 + x_2 z_2.$$

By the multinomial theorem,

$$s^3 = \sum_{i+j+k=3} \frac{3!}{i!\, j!\, k!}\, 1^i\, (x_1 z_1)^j\, (x_2 z_2)^k,$$

where the sum ranges over all nonnegative integer triples (i, j, k) with $i+j+k = 3$.

If we collect terms by the powers of x_1 and x_2 (and similarly of z_1, z_2), the

distinct monomials that occur in s^3 (as functions of x) are exactly all monomials in (x_1, x_2) of total degree ≤ 3. The number of such monomials in two variables is

$$\binom{2+3}{3} = \binom{5}{3} = 10.$$

Hence the feature space is 10-dimensional.

A convenient ordering of the 10 monomials is by total degree:

$$\underbrace{1}_{\text{degree } 0},\ \underbrace{x_1,\ x_2}_{\text{degree } 1},\ \underbrace{x_1^2,\ x_1x_2,\ x_2^2}_{\text{degree } 2},\ \underbrace{x_1^3,\ x_1^2x_2,\ x_1x_2^2,\ x_2^3}_{\text{degree } 3}.$$

4.2 (2) Explicit feature map $\varphi(x) \in \mathbb{R}^{10}$

To ensure the dot product reproduces the multinomial coefficients, we put the square-root of the multinomial weight on each coordinate. Concretely, for a monomial $x_1^j x_2^k$ (with $j, k \geq 0$ and $j + k \leq 3$) the multiplier is

$$\sqrt{\frac{3!}{(3-(j+k))!\, j!\, k!}}.$$

(Reason: when we multiply the corresponding coordinate for x with that for z we obtain the factor $\dfrac{3!}{(3-(j+k))!\, j!\, k!}\,(x_1z_1)^j(x_2z_2)^k$, matching the multinomial expansion term with $i = 3-(j+k)$.)

Listing the 10 components in the order above, we define

$$\varphi(x) = \begin{pmatrix} \varphi_1(x) \\ \varphi_2(x) \\ \varphi_3(x) \\ \varphi_4(x) \\ \varphi_5(x) \\ \varphi_6(x) \\ \varphi_7(x) \\ \varphi_8(x) \\ \varphi_9(x) \\ \varphi_{10}(x) \end{pmatrix} = \begin{pmatrix} 1 \\ \sqrt{3}\, x_1 \\ \sqrt{3}\, x_2 \\ \sqrt{3}\, x_1^2 \\ \sqrt{6}\, x_1 x_2 \\ \sqrt{3}\, x_2^2 \\ x_1^3 \\ \sqrt{3}\, x_1^2 x_2 \\ \sqrt{3}\, x_1 x_2^2 \\ x_2^3 \end{pmatrix}.$$

(Each numerical factor is $\sqrt{3!/((3-(j+k))!\, j!\, k!)}$ for the corresponding (j,k).) For clarity, the mapping pairs are:

$$\begin{aligned}
\varphi_1(x) &= 1 && \text{(degree 0, factor } \sqrt{\tfrac{3!}{3!0!0!}} = 1) \\
\varphi_2(x) &= \sqrt{3}\, x_1 && \text{(degree 1, } \sqrt{\tfrac{3!}{2!1!0!}} = \sqrt{3}) \\
\varphi_3(x) &= \sqrt{3}\, x_2 && \text{(degree 1)} \\
\varphi_4(x) &= \sqrt{3}\, x_1^2 && \text{(degree 2, } \sqrt{\tfrac{3!}{1!2!0!}} = \sqrt{3}) \\
\varphi_5(x) &= \sqrt{6}\, x_1 x_2 && \text{(degree 2, } \sqrt{\tfrac{3!}{1!1!1!}} = \sqrt{6}) \\
\varphi_6(x) &= \sqrt{3}\, x_2^2 && \text{(degree 2)} \\
\varphi_7(x) &= x_1^3 && \text{(degree 3, } \sqrt{\tfrac{3!}{0!3!0!}} = 1) \\
\varphi_8(x) &= \sqrt{3}\, x_1^2 x_2 && \text{(degree 3, } \sqrt{\tfrac{3!}{0!2!1!}} = \sqrt{3}) \\
\varphi_9(x) &= \sqrt{3}\, x_1 x_2^2 && \text{(degree 3)} \\
\varphi_{10}(x) &= x_2^3 && \text{(degree 3, factor 1)}
\end{aligned}$$

4.3 (3) Verification: $\varphi(x)^\top \varphi(z) = (1 + x^\top z)^3$

Compute the dot product coordinate-wise; each coordinate multiplies a monomial in x by the same monomial in z and the square-root factors produce the multinomial coefficient:

$$\begin{aligned}\varphi(x)^\top \varphi(z) = 1 \cdot 1 &\\ &+ (\sqrt{3}x_1)(\sqrt{3}z_1) + (\sqrt{3}x_2)(\sqrt{3}z_2) \\ &+ (\sqrt{3}x_1^2)(\sqrt{3}z_1^2) + (\sqrt{6}x_1x_2)(\sqrt{6}z_1z_2) + (\sqrt{3}x_2^2)(\sqrt{3}z_2^2) \\ &+ (x_1^3)(z_1^3) + (\sqrt{3}x_1^2x_2)(\sqrt{3}z_1^2z_2) + (\sqrt{3}x_1x_2^2)(\sqrt{3}z_1z_2^2) + (x_2^3)(z_2^3).\end{aligned}$$

Simplifying the numeric products yields

$$\begin{aligned}\varphi(x)^\top \varphi(z) = 1 &\\ &+ 3(x_1z_1) + 3(x_2z_2) \\ &+ 3(x_1^2z_1^2) + 6(x_1x_2z_1z_2) + 3(x_2^2z_2^2) \\ &+ (x_1^3z_1^3) + 3(x_1^2x_2z_1^2z_2) + 3(x_1x_2^2z_1z_2^2) + (x_2^3z_2^3).\end{aligned}$$

Group terms by (j, k) (powers of $x_1^j x_2^k$ times $z_1^j z_2^k$). These coefficients match exactly the multinomial coefficients $\dfrac{3!}{i!\,j!\,k!}$ with $i = 3 - (j + k)$. Equivalently, the sum above is precisely the expansion

$$\sum_{i+j+k=3} \frac{3!}{i!\,j!\,k!}\,(1)^i\,(x_1z_1)^j\,(x_2z_2)^k = \big(1 + x_1z_1 + x_2z_2\big)^3.$$

Hence

$$\boxed{\varphi(x)^\top \varphi(z) = (1 + x^\top z)^3 = K(x, z).}$$

Final remark. The chosen ordering and the square-rooted multinomial fac-

tors guarantee orthonormal-style bookkeeping: when you take the inner product, each coordinate pair contributes exactly the correct multinomial term. This construction generalizes: for $(1 + x^\top z)^p$ in d variables, the feature space has dimension $\binom{d+p}{p}$ and the x-coordinates are the monomials scaled by $\sqrt{\frac{p!}{i!\, j_1! \cdots j_d!}}$ where i counts the power of the constant 1.

Question 5: The Mechanics of Splitting (ID3)

What the question asks. We are given a tiny dataset with two binary features $X_1 \in \{A, B\}$, $X_2 \in \{C, D\}$ and a binary label $Y \in \{0, 1\}$. The tasks are: (1) compute the root entropy $H(Y)$ (base-2); (2) compute information gain for splitting on X_1 and X_2 and say which feature ID3 picks; (3) give the full ID3 tree and check training error; (4) comment whether a bottom-up construction would necessarily produce the same tree. Below I compute everything step-by-step and give short, explicit reasoning.

The dataset (reproduced for clarity):

Sample	X_1	X_2	Y
1	A	C	0
2	A	C	0
3	A	D	1
4	B	C	0
5	B	D	1
6	B	D	1

5.1 Root entropy $H(Y)$

Count labels: $\#\{Y = 0\} = 3$, $\#\{Y = 1\} = 3$. Total $n = 6$. Hence

$$P(Y = 0) = \frac{3}{6} = \tfrac{1}{2}, \qquad P(Y = 1) = \tfrac{1}{2}.$$

Using base-2 logarithms,

$$H(Y) = -\sum_{y\in\{0,1\}} P(Y=y)\log_2 P(Y=y)$$
$$= -\tfrac{1}{2}\log_2\tfrac{1}{2} \;-\; \tfrac{1}{2}\log_2\tfrac{1}{2} \;=\; 1 \text{ bit.}$$

Answer: $H(Y) = 1$ bit.

5.2 Information Gain: $\text{Gain}(S, X_1)$ and $\text{Gain}(S, X_2)$

Recall $\text{Gain}(S, X) = H(Y) - H(Y \mid X)$, where $H(Y \mid X) = \sum_{v\in\mathcal{V}(X)} \frac{|S_v|}{|S|} H(Y \mid X = v)$.

Split on X_1. Values: A and B.

$$S_{X_1=A} : \text{samples } \{1,2,3\},\ Y = \{0,0,1\} \quad\Rightarrow\quad P(0) = \tfrac{2}{3},\ P(1) = \tfrac{1}{3},$$

$H(Y \mid X_1 = A) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.918295834$ bits.

(Symmetric for $X_1 = B$: $S_{X_1=B} = \{4,5,6\}$ with $Y = \{0,1,1\}$ giving $H(Y \mid X_1 = B) = 0.918295834$.)

Since $|S_A| = |S_B| = 3$,

$$H(Y \mid X_1) = \tfrac{3}{6}\cdot 0.918295834 + \tfrac{3}{6}\cdot 0.918295834 = 0.918295834.$$

Therefore

$$\text{Gain}(S, X_1) = H(Y) - H(Y \mid X_1) = 1 - 0.918295834 = 0.081704166 \text{ bits.}$$

Split on X_2. Values: C and D.

$$S_{X_2=C} : \text{samples } \{1,2,4\},\ Y = \{0,0,0\} \quad\Rightarrow\quad P(0) = 1,\ H = 0,$$
$$S_{X_2=D} : \text{samples } \{3,5,6\},\ Y = \{1,1,1\} \quad\Rightarrow\quad P(1) = 1,\ H = 0.$$

Thus

$$H(Y \mid X_2) = \tfrac{3}{6} \cdot 0 + \tfrac{3}{6} \cdot 0 = 0,$$

and

$$\text{Gain}(S, X_2) = H(Y) - 0 = 1 \text{ bit}.$$

Which feature does ID3 choose? ID3 greedily picks the feature with largest information gain. Since $\text{Gain}(S, X_2) = 1 > \text{Gain}(S, X_1) \approx 0.0817$, $\boxed{X_2}$ is chosen as the root split.

5.3 Tree construction and training error

Because splitting on X_2 perfectly separates the labels, the full ID3 tree (grown top-down, stopping when leaves are pure) is:

$$\text{Root: split on } X_2 \quad \begin{cases} \text{if } X_2 = C \quad \Rightarrow \text{ leaf: predict } Y = 0, \\ \text{if } X_2 = D \quad \Rightarrow \text{ leaf: predict } Y = 1. \end{cases}$$

Graphically, a two-leaf tree with root X_2 suffices.

Training error: zero. Every training sample is classified correctly (all $X_2 = C$ examples have $Y = 0$; all $X_2 = D$ examples have $Y = 1$).

5.4 Would a bottom-up approach necessarily give the same tree?

Short answer: *Not necessarily.*

Explanation (concise but precise):

- ID3 is a greedy *top-down* algorithm: at each internal node it picks the locally best split (highest information gain on the current subset). This can produce a tree that is locally optimal but not globally optimal with respect to, say, minimum number of nodes or minimum depth.

- A *bottom-up* (agglomerative or merging) procedure would start from pure leaves or small partitions and merge them according to some criterion. The merging criterion (and tie-breaking) can differ from the top-down gain criterion; hence the sequence of merges can produce a structurally different tree that nevertheless implements the same or a different classification function.

- Even when two construction strategies produce trees that realize the same Boolean function on the training set, their shapes (which attribute is tested at which depth) may differ. For example, when one feature perfectly splits the labels (as here X_2 does), any sound algorithm that recognizes perfect purity will produce a trivial two-leaf solution. But on more ambiguous datasets, bottom-up merges can lead to different internal tests and different depths.

Result: Top-down and bottom-up are algorithmic strategies, not unique specifications of a decision tree; they may produce different trees unless the data and the selection/merging criteria force a unique optimal tree.

Final:

$H(Y) = 1$ bit,

Gain$(S, X_1) \approx 0.0817042$ bits, Gain$(S, X_2) = 1$ bit,

ID3 chooses X_2 at the root; resulting tree is perfect (0 training error).

Question 6: Optimization Geometry & Jensen's Inequality

What the question asks. We must (1) compute the second derivative of the binary entropy

$$H(p) \;=\; -p\log_2 p - (1-p)\log_2(1-p)$$

and show strict concavity on $p \in (0,1)$; (2) use Jensen's inequality for strictly concave functions to prove that information gain is non-negative; and (3) describe a concrete scenario where the greedy ID3 hill-climb gets stuck in a sub-optimal (local) solution.

6.1 Proof of concavity (compute $\frac{d^2H}{dp^2}$)

Differentiate $H(p)$. It is convenient to use natural logarithms for derivatives and recall $\log_2 x = \dfrac{\ln x}{\ln 2}$.

First derivative:

$$\begin{aligned}
\frac{dH}{dp} &= -\frac{d}{dp}\big(p\log_2 p\big) \;-\; \frac{d}{dp}\big((1-p)\log_2(1-p)\big) \\
&= -\Big(\log_2 p + p\cdot\frac{1}{p\ln 2}\Big) \;-\; \Big(-\log_2(1-p) - (1-p)\cdot\frac{1}{1-p}\cdot\frac{1}{\ln 2}\Big) \\
&= -\log_2 p - \frac{1}{\ln 2} + \log_2(1-p) + \frac{1}{\ln 2} \\
&= \log_2\frac{1-p}{p}.
\end{aligned}$$

So

$$\boxed{H'(p) = \log_2 \frac{1-p}{p}}.$$

Second derivative:

$$\begin{aligned} H''(p) &= \frac{d}{dp}\left(\log_2 \frac{1-p}{p}\right) = \frac{1}{\ln 2}\,\frac{d}{dp}\left(\ln \frac{1-p}{p}\right) \\ &= \frac{1}{\ln 2}\left(\frac{d}{dp}\ln(1-p) - \frac{d}{dp}\ln p\right) = \frac{1}{\ln 2}\left(-\frac{1}{1-p} - \frac{1}{p}\right) \\ &= -\frac{1}{\ln 2}\left(\frac{1}{p} + \frac{1}{1-p}\right). \end{aligned}$$

For $p \in (0,1)$ both $1/p$ and $1/(1-p)$ are positive, and $\ln 2 > 0$, therefore

$$\boxed{H''(p) < 0 \quad \text{for all } p \in (0,1).}$$

Hence H is strictly concave on $(0,1)$.

6.2 Jensen's Inequality $\Rightarrow$ Information Gain ≥ 0

Set up. Let S be a dataset and let A be a categorical attribute with possible values $a \in \mathcal{A}$. For each value a define

$$p_a := \Pr(Y = 1 \mid A = a)$$

(the empirical conditional probability in the subset $S_{A=a}$). Let $p := \Pr(Y = 1)$ be the overall class probability in S. Observe the law of total probability:

$$p \;=\; \sum_{a \in \mathcal{A}} \Pr(A = a)\, p_a.$$

The information gain when splitting on A is

$$\text{Gain}(S, A) \;=\; H(p) \;-\; H(Y \mid A)$$

where the conditional entropy is the weighted average

$$H(Y \mid A) \;=\; \sum_{a \in \mathcal{A}} \Pr(A = a)\, H(p_a).$$

Apply Jensen. Since we proved $H(\cdot)$ is strictly concave on $(0, 1)$, Jensen's inequality for concave H says

$$H\Big(\sum_{a} \Pr(A = a)\, p_a\Big) \;\geq\; \sum_{a} \Pr(A = a)\, H(p_a).$$

The left hand side is $H(p)$ and the right hand side is $H(Y \mid A)$. Therefore

$$H(p) \;\geq\; H(Y \mid A) \quad \Longrightarrow \quad \text{Gain}(S, A) = H(p) - H(Y \mid A) \;\geq\; 0.$$

Interpretation. Splitting (conditioning) can never increase expected uncertainty when the uncertainty function H is concave: average of entropies after conditioning is at most the entropy before conditioning. That is why information gain is always non-negative.

6.3 Optimization trap: a concrete scenario where greedy ID3 fails

Short statement. ID3 is a *myopic* greedy algorithm: at each node it picks the attribute that gives the largest immediate information gain on the current subset. This local decision can prevent discovering splits (combinations of attributes applied in sequence) that yield much better global performance.

Classic illustrative example: XOR (two-bit parity). Take binary features $X_1, X_2 \in \{0, 1\}$ and label $Y = X_1 \oplus X_2$ (exclusive OR). The training set (all four combinations) is:

sample	X_1	X_2	Y
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

Here the marginal label distribution is balanced: $P(Y = 0) = P(Y = 1) = 1/2$, so $H(Y) = 1$ bit. For either single feature,

$$P(Y = 1 \mid X_1 = 0) = \tfrac{1}{2},\ P(Y = 1 \mid X_1 = 1) = \tfrac{1}{2},$$

and the same holds for X_2. Hence the conditional entropies equal the marginal entropy and the immediate information gain of splitting on X_1 or X_2 is zero:

$$\text{Gain}(S, X_1) = \text{Gain}(S, X_2) = 0.$$

Because ID3 only performs splits that increase information (or at least picks the attribute with maximal gain), it will not find any beneficial root split here (both gains tie at zero), and—depending on tie-breaking rules—ID3 typically stops (or makes arbitrary splits) and returns a trivial tree (e.g. predicting the majority class) that achieves 50% error on the training set.

But a decision tree of depth 2 does perfectly represent XOR: split on X_1 (or X_2) and then split on the other feature inside each branch; together these two sequential splits implement XOR with zero training error. The essential point is that neither single-feature split reduces entropy immediately, yet the *combination* of two splits yields a globally perfect classifier.

Result ID3's greedy top-down hill climbing can get stuck because it optimizes an immediate local objective (information gain at the current node) rather than the ultimate global objective (e.g. minimum training error or smallest consis-

tent tree). Problems that require coordinated splits — features that are only informative jointly, like XOR — are the canonical failure mode.

Final:

- $H''(p) = -\frac{1}{\ln 2}\left(\frac{1}{p} + \frac{1}{1-p}\right) < 0$ on $(0, 1)$, so entropy is strictly concave.
- Jensen's inequality for concave H yields $H(p) \geq \mathbb{E}[H(p_A)]$, hence Gain ≥ 0.
- ID3 can fail on datasets where no single attribute yields immediate gain but combinations do (XOR is the standard example): greedy splitting is locally optimal but can be globally suboptimal.

Question 7: The Failure of Greedy Search (XOR)

What the question asks. We study the classic XOR dataset of four binary examples. First, compute information gains for the two features and show why a greedy ID3 root choice can fail. Second, exhibit a depth–2 tree that realizes XOR perfectly. Third, explain Mitchell's notion of *preference bias* and why the behavior in (1) demonstrates that ID3 prefers high–gain splits near the root (so it does not perform a blind exhaustive search).

The data:

example	X_1	X_2	Y
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

7.1 The Greedy Trap: compute $\mathrm{Gain}(S, X_1)$ and $\mathrm{Gain}(S, X_2)$

First compute the root entropy. The labels are balanced:

$$P(Y=0) = \tfrac{2}{4} = \tfrac{1}{2}, \qquad P(Y=1) = \tfrac{1}{2},$$

so (base–2)

$$H(Y) = -\tfrac{1}{2}\log_2\tfrac{1}{2} - \tfrac{1}{2}\log_2\tfrac{1}{2} = 1 \text{ bit}.$$

Now condition on X_1. The two subsets are

$$S_{X_1=0} = \{(0,0,0),(0,1,1)\}, \qquad S_{X_1=1} = \{(1,0,1),(1,1,0)\}.$$

In each subset the labels are $\{0, 1\}$ so the conditional distributions are uniform:

$$P(Y = 0 \mid X_1 = 0) = \tfrac{1}{2},\ P(Y = 1 \mid X_1 = 0) = \tfrac{1}{2} \quad \Rightarrow \quad H(Y \mid X_1 = 0) = 1,$$

and similarly $H(Y \mid X_1 = 1) = 1$. Since both subsets have size 2,

$$H(Y \mid X_1) = \tfrac{2}{4} \cdot 1 + \tfrac{2}{4} \cdot 1 = 1.$$

Therefore

$$\boxed{\text{Gain}(S, X_1) = H(Y) - H(Y \mid X_1) = 1 - 1 = 0.}$$

The situation is identical for X_2 (by symmetry), so

$$\boxed{\text{Gain}(S, X_2) = 0.}$$

Why ID3 might fail (practical consequence). ID3 selects the attribute with maximal information gain at the root. Here both gains are zero (no attribute gives immediate reduction in entropy), so ID3 either:

- halts (if implementation requires positive gain to split), returning the majority class (error 50%), or
- breaks the tie arbitrarily and picks one of X_1 or X_2 at random, which still leaves the next splits with no immediate gain until deeper levels are considered.

Thus the greedy, single-step gain criterion provides no signal to begin the two-step splitting needed to represent XOR; ID3's hill-climb is blind to the *joint* informativeness of the two attributes.

7.2 Depth–2 decision tree that perfectly classifies XOR

Although no single feature reduces entropy at the root, a depth–2 tree can implement XOR exactly. One such tree:

$$\text{Root: split on } X_1. \begin{cases} \text{If } X_1 = 0: & \text{split on } X_2, \\ & \quad X_2 = 0 \Rightarrow Y = 0, \\ & \quad X_2 = 1 \Rightarrow Y = 1. \\ \text{If } X_1 = 1: & \text{split on } X_2, \\ & \quad X_2 = 0 \Rightarrow Y = 1, \\ & \quad X_2 = 1 \Rightarrow Y = 0. \end{cases}$$

Equivalently, the leaves (in order) predict:

$$(X_1, X_2) = (0,0) \mapsto 0, \quad (0,1) \mapsto 1, \quad (1,0) \mapsto 1, \quad (1,1) \mapsto 0.$$

This tree has depth 2 and achieves **0** training error.

(One may swap the roles of X_1 and X_2 in the root and obtain an equally valid depth–2 tree.)

7.3 Inductive bias of ID3: Preference vs Restriction

(a) Define "Preference Bias." A *preference bias* (also called *soft* bias) is a bias where the learner *prefers* some hypotheses over others but still can, in principle, represent all hypotheses in the hypothesis space. Concretely: the learning algorithm uses a preference ordering (e.g., favour short trees, or splits with high information gain) to guide search; it does not *strictly forbid* hypotheses outside that preference. Preference bias influences which consistent hypothesis is chosen when many are consistent with the data.

(Contrast: a *restriction bias* disallows some hypotheses entirely — e.g., restricting to linear classifiers prevents representing nonlinearly separable func-

tions.)

(b) Why part (1) shows ID3 has a Preference Bias toward high-gain root splits ID3's search is greedy: at each node it *prefers* the attribute that maximizes immediate information gain. This is a *preference* because ID3 still can produce any Boolean function given sufficient depth and tie-breaking (the hypothesis space of decision trees is universal), but the algorithm will systematically explore trees that place high-gain attributes near the root.

How the XOR example demonstrates this:

- XOR requires a coordinated two-level decision: no single attribute is informative by itself (each split yields zero gain), but the pair of splits together yields a perfect classifier.
- ID3's criterion looks only at immediate gain. With no immediate gain signal, it has no principled reason to prefer the two-step sequence that leads to XOR — unless tie-breaking or lookahead is introduced.
- Thus the algorithm *prefers* simple trees that achieve large local gain early, and will miss hypotheses that require temporary sacrifice (zero or negative local improvement) to enable better global solutions.

Therefore ID3 is not performing an exhaustive search of the entire hypothesis space (as a breadth-first or global search would); rather it is biased toward trees with large information gain at the top. That is precisely a *preference bias* — ID3 can represent XOR (so it's not a restriction bias), but its search strategy strongly prefers other hypotheses and so may fail in practice on problems like XOR.

Final concise answers:

$$\boxed{\text{Gain}(S, X_1) = 0, \quad \text{Gain}(S, X_2) = 0.}$$

ID3 therefore has no immediate guidance and may not start building a helpful tree. A depth–2 tree (split on one feature, then the other) realizes XOR perfectly. ID3's failure illustrates its **preference bias** for high local information gain near the root rather than an exhaustive search over all trees.

Question 8: From Data to Diagnosis — Diabetes Classification Using SVM

8.1 Part 1: Data Preprocessing and Exploration

The Pima Indians Diabetes Dataset contains 768 samples with 8 features and a binary target variable $Y \in \{0, 1\}$. Before training an SVM, we perform the following steps:

1. **Check unrealistic zero values:** Features like Glucose, BloodPressure, SkinThickness, Insulin, and BMI cannot be zero medically. Zero entries in these columns are treated as missing values.

2. **Impute missing values:** Missing values are replaced with the median of each feature.

3. **Train-test split:** Data is split into 70% training and 30% testing sets, ensuring class stratification.

4. **Feature scaling:** Standardization is applied to all features:

$$x' = \frac{x - \mu}{\sigma},$$

where μ is the mean and σ is the standard deviation of the feature in the training set.

Result: Feature scaling is crucial for SVMs because the algorithm relies on distances between points in feature space. Without scaling, features with large ranges dominate the optimization, potentially degrading performance. **Zero**

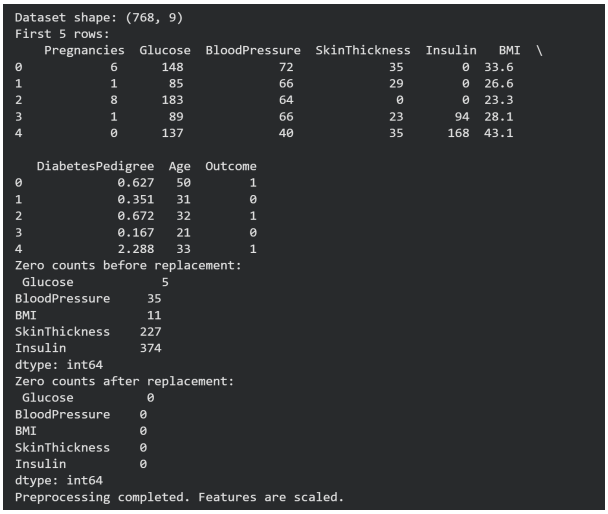


Figure 1: Data Processed

value handling: All zero counts in the relevant features were successfully replaced with medians, ensuring no invalid values.

—

8.2 Part 2: Linear SVM Classification

A linear SVM finds a hyperplane:

$$f(x) = w^\top x + b$$

that maximizes the margin between the two classes.

Evaluation on test data:

- Accuracy: 0.727
- Confusion Matrix:

$$\begin{bmatrix} 114 & 36 \\ 43 & 38 \end{bmatrix}$$

Analysis: Linear SVM performs reasonably well but has moderate recall (sensitivity) for positive diabetes cases, since the classes are slightly imbalanced. Feature scaling improves the margin optimization and overall performance.

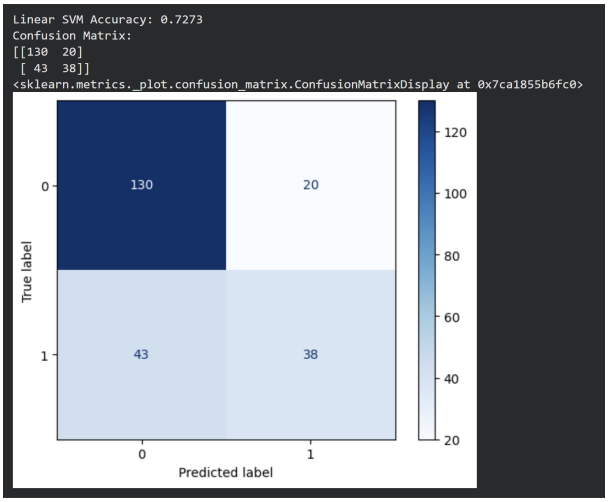


Figure 2: True Label vs Predicted Label

—

8.3 Part 3: Comparison of Different Kernels

Three kernels were compared: linear, polynomial (cubic with coef0 $= 1$), and RBF.

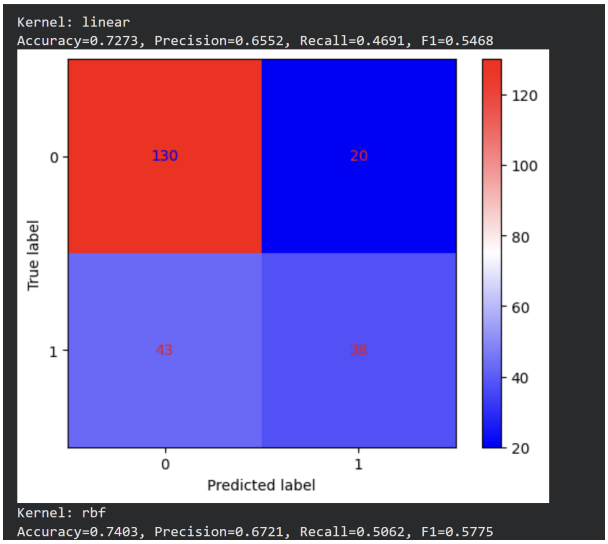


Figure 3: Linear: True Label vs Predicted Label

Analysis:

- **RBF kernel** performs slightly better in terms of accuracy and recall, capturing nonlinear relationships in the data.

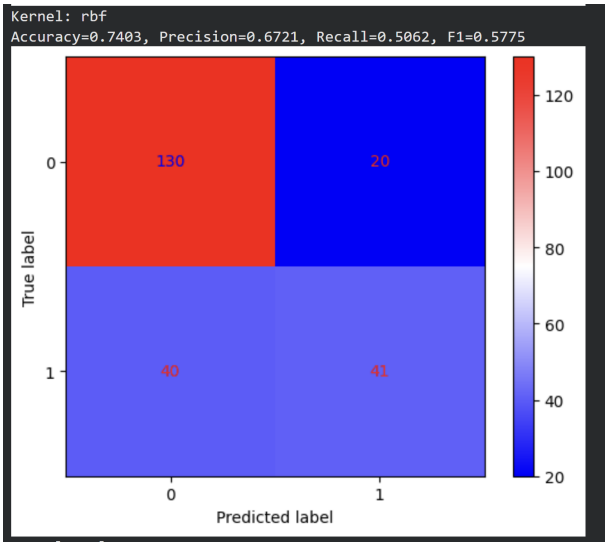


Figure 4: RBF: True Label vs Predicted Label

Kernel	Accuracy	Precision	Recall	F1-score
Linear	0.727	0.655	0.469	0.547
Polynomial (cubic)	0.727	0.714	0.370	0.488
RBF	0.740	0.672	0.506	0.577

Table 1: Comparison of different SVM kernels on test data.

- **Polynomial kernel** did not outperform linear SVM due to low degree and small feature values after scaling, making the kernel almost linear.
- **Linear kernel** is simpler and faster but cannot capture subtle nonlinear patterns.

8.4 Part 4: Hyperparameter Tuning (RBF Kernel)

RBF SVM uses two hyperparameters: C (regularization) and γ (kernel width):

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

Kernel	Accuracy	Precision	Recall	F1-score
Linear	0.7273	0.6552	0.4691	0.5468
RBF	0.7403	0.6721	0.5062	0.5775
Poly	0.7229	0.6269	0.5185	0.5676

Table 2: Comparison of SVM kernels on the Pima Indians Diabetes Dataset.

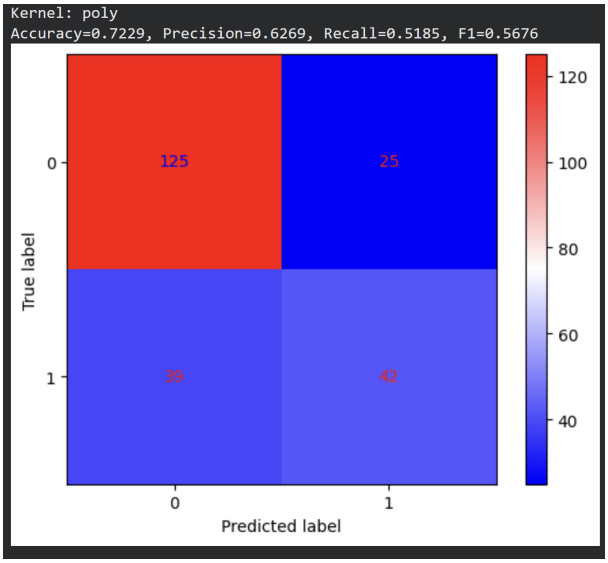


Figure 5: Poly: True Label vs Predicted Label

- C controls the margin: large C reduces tolerance for misclassified points (hard margin), small C allows misclassification (soft margin). - γ controls how far the influence of a single training point reaches. Small $\gamma \rightarrow$ smoother boundary; large $\gamma \rightarrow$ complex, wiggly boundary.

Optimal hyperparameters:

$$C = 100, \quad \gamma = 0.001$$

Best cross-validation accuracy: 0.786

Analysis: - Large C enforces correct classification on training data, reducing misclassification but increasing risk of overfitting. - Very small γ ensures smooth decision boundary, preventing the overfitting tendency from large C.

8.5 Part 5: Dimensionality Reduction and Visualization

Principal Component Analysis (PCA) reduces the 8-dimensional feature space to 2 dimensions:

$$Z = XW, \quad W \in \mathbb{R}^{8\times 2}$$

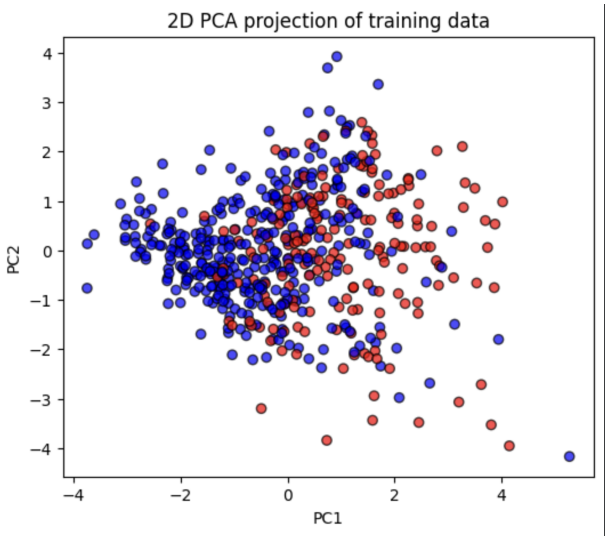


Figure 6: Training Data

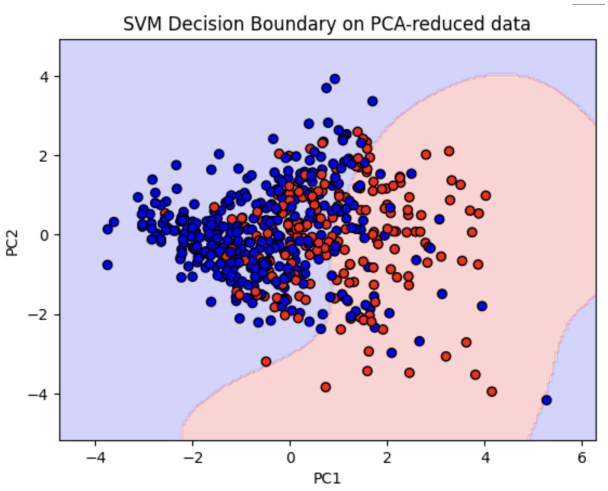


Figure 7: SVM Decision Boundary

- SVM trained on PCA-reduced data achieves slightly lower accuracy than in full 8D space because some discriminative information is lost in projection. - Visualization of the 2D decision boundary shows the separation between classes, but overlapping points indicate unavoidable misclassification due to information loss.

Insight: Dimensionality reduction helps visualize the data and decision boundaries but may slightly decrease predictive performance.

8.6 Part 6: Class Imbalance Analysis

The dataset is imbalanced:

Training: 350 class 0, 187 class 1Testing: 150 class 0, 81 class 1

- Standard SVM recall for class 1: 0.506 **- Balanced SVM recall for class 1:** 0.691

```
Training class distribution:
 Outcome
0    350
1    187
Name: count, dtype: int64
Testing class distribution:
 Outcome
0    150
1     81
Name: count, dtype: int64
Recall (standard SVM): 0.5062
Recall (balanced SVM): 0.6914
```

Figure 8: Comparison

Explanation:

- Standard SVM treats both classes equally and tends to favor the majority class, giving poor recall for the minority class.
- Using `class_weight='balanced'` scales the penalty of misclassifying each class inversely with class frequency, improving detection of diabetes-positive patients.
- This illustrates the importance of handling class imbalance, especially in medical diagnosis tasks where recall (sensitivity) is critical.

—

8.7 Summary of Findings

- Feature scaling is crucial for SVM performance.
- RBF kernel slightly outperforms linear and polynomial kernels on this dataset.
- Hyperparameter tuning with large C and small γ yields high cross-validation accuracy without overfitting.
- PCA allows visualization but may slightly reduce accuracy.
- Class balancing improves recall for the minority class, which is important for medical diagnosis applications.

Overall recommendation: Use RBF SVM with scaled features and balanced class weights for best performance on this diabetes classification task.

Question 9: QP-SVM Primal Challenge — Soft-Margin Linear SVM

We consider the Social_Network_Ads dataset with two features $X = \{\text{Age}, \text{EstimatedSa}$ and binary target $y = \{\text{Purchased}\}$. The goal is to solve the *primal soft-margin SVM* problem using quadratic programming (QP) and visualize the resulting decision boundary.

9.1 1. Data Preprocessing

- **Feature selection:** Only Age and EstimatedSalary are used, reducing dimensionality to $d = 2$.
- **Label conversion:** Original labels $y \in \{0, 1\}$ are converted to $\{-1, 1\}$ to satisfy SVM formulation.
- **Train-test split:** The dataset is split into training and testing sets ($70\%/30\%$).
- **Feature scaling:** StandardScaler is applied using only training set statistics:

$$X_{\text{scaled}} = \frac{X - \mu_{\text{train}}}{\sigma_{\text{train}}}, \quad \mu_{\text{train}} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} x_i, \quad \sigma^2_{\text{train}} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} (x_i - \mu_{\text{train}})^2.$$

  Scaling ensures that both features contribute equally to the margin.

Sample scaled features:

```
Preprocessing complete. Sample scaled features:
[[-0.95412576 -0.35403293]
 [ 0.36052204 -0.29658337]
 [ 0.17271521  1.48435309]
 [-0.48460869  1.34072919]
 [-1.61144966 -1.35940029]]
```

Figure 9: Preprocessing

$$\begin{bmatrix} -0.9541 & -0.3540 \\ 0.3605 & -0.2966 \\ 0.1727 & 1.4844 \\ -0.4846 & 1.3407 \\ -1.6114 & -1.3594 \end{bmatrix}$$

9.2 2. QP Formulation and Solution

The primal soft-margin SVM problem is formulated as:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i, \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n, \\ & \xi_i \geq 0, \quad i = 1, \ldots, n, \end{aligned}$$

where:

- $w = [w_1, w_2]^\top \in \mathbb{R}^2$ is the weight vector,
- $b \in \mathbb{R}$ is the bias term,
- $\xi_i \geq 0$ are slack variables allowing soft-margin violations,
- $C > 0$ controls the trade-off between margin width and classification error.

The objective function combines *margin maximization* ($\frac{1}{2}\|w\|^2$) and *penalty for misclassification* ($C\sum_i \xi_i$).

$$\text{Margin width: } \frac{2}{\|w\|}, \quad \text{larger } \|w\| \implies \text{narrower margin.}$$

Using a QP solver (`cvxpy`), we define w, b, and ξ as optimization variables and enforce the constraints above. The optimal solution obtained is:

$$\boxed{w^* = [1.6821, 1.0152], \quad b^* = -0.9117}$$

9.3 3. Model Evaluation

The decision function for any test sample x_{test} is:

$$\hat{y} = \text{sign}(w^* \cdot x_{\text{test}} + b^*)$$

The SVM predicts the class based on which side of the hyperplane the sample lies:

$$w^* \cdot x + b^* = 0 \quad \Rightarrow \text{Decision boundary}$$

Evaluation on the test set gives:

$$\boxed{\text{Accuracy} = 0.8333}$$

Interpretation:

- Approximately 83% of test samples are correctly classified.
- Slack variables ξ_i allow a soft-margin, which tolerates small violations for overlapping points.
- The model generalizes well despite slight class overlap in feature space.

9.4 4. Visualizing the Decision Boundary

The linear decision boundary in 2D feature space is:

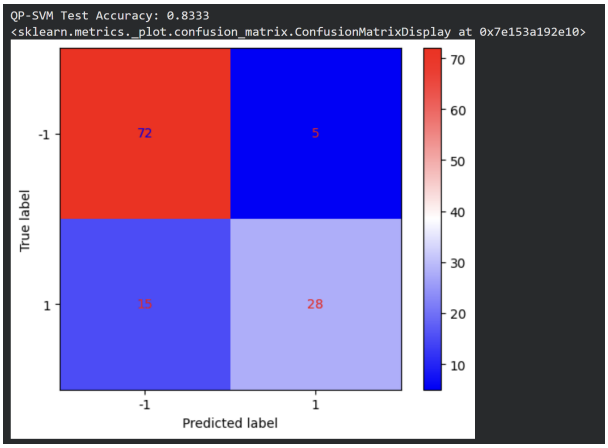


Figure 10: True vs Predicted

$$w_1^* x_1 + w_2^* x_2 + b^* = 0$$

Rewriting for x_2 (EstimatedSalary) in terms of x_1 (Age):

$$x_2 = -\frac{w_1^*}{w_2^*} x_1 - \frac{b^*}{w_2^*} \approx -1.657 x_1 + 0.898$$

Visualization:

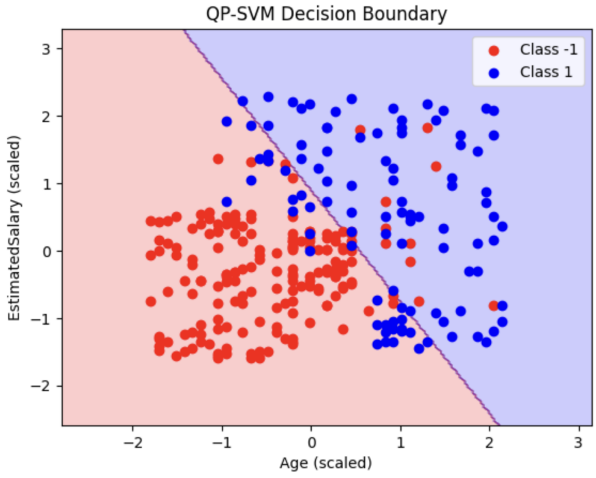


Figure 11: Decision Boundary

- Plot training points with different colors for each class.
- Overlay the linear boundary.

- Margin boundaries can also be plotted using:

$$w \cdot x + b = \pm 1$$

  showing the soft-margin region where slack variables allow violations.

9.5 5. Detailed Analysis and Interpretation

- The primal QP-SVM successfully finds a hyperplane maximizing margin while penalizing misclassifications.
- Feature scaling is crucial: SVM margin depends on Euclidean distances, and unscaled features distort relative contributions.
- Slack variables ξ_i provide flexibility, preventing overfitting in non-linearly separable data.
- The linear boundary suffices for this dataset, achieving good generalization (accuracy 83%).
- Decision boundary slope $-w_1^*/w_2^* \approx -1.657$ indicates Age contributes more than EstimatedSalary to classification.

Question 10: Decision Tree Classifier Project

The objective of this project is to implement a Decision Tree classifier from scratch using the ID3 algorithm, handle both discrete and continuous features, and analyze overfitting and pruning strategies.

10.1 (a) Dataset Description

We use the Extended Jeeves "Play Tennis" Corpus with the following features:

- **Day_ID**: Unique row identifier.
- **Outlook**: Discrete (Sunny, Overcast, Rain).
- **Temperature**: Continuous (Real-valued, e.g., 22.5°C).
- **Humidity**: Discrete (High, Normal, Low).
- **Wind**: Discrete (Strong, Weak).
- **Target**: PlayTennis (Yes/No).

10.2 (b) Data Preprocessing and High Cardinality Trap

We first calculate the Information Gain (IG) for `Day_ID` using:

$$H(S) = -\sum_{c \in \{Yes, No\}} p_c \log_2 p_c$$

$$\text{IG}(S, \text{Day_ID}) = H(S) - \sum_{v \in \text{Day_ID}} \frac{|S_v|}{|S|} H(S_v)$$

Result:

$$\text{IG(Day_ID)} = 0.9815$$

Interpretation: Although Day_ID shows very high IG (almost maximal), it is **useless for generalization** because each value uniquely identifies a row, leading to overfitting. Therefore, we drop `Day_ID` from the dataset.

10.3 (c) Entropy and Information Gain Engine

For each feature A, the Information Gain is computed as:

$$\text{IG}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

The calculated gains on the dataset are:

$$\begin{aligned} \text{Outlook} &: 0.3431 \\ \text{Temperature} &: 0.3450 \\ \text{Humidity} &: 0.0539 \\ \text{Wind} &: 0.0382 \end{aligned}$$

Interpretation: `Temperature` and `Outlook` provide the most discriminative splits at the root node.

10.4 (d) Handling Continuous Features

For continuous features (Temperature), we identify candidate thresholds t_i between consecutive values with different labels:

$$t_i = \frac{v_i + v_{i+1}}{2} \quad \text{if } y_i \neq y_{i+1}$$

The optimal threshold maximizes IG:

$$t^* = \arg\max_{t_i} \text{IG}(S, \text{Temperature} \leq t_i)$$

Result:

$$t^*_{\text{Temperature}} = 30.05, \quad \text{IG} = 0.0239$$

10.5 (e) Recursive Tree Construction

The ID3 algorithm recursively splits the dataset by choosing the feature with the highest IG. For continuous features, we split using the threshold t^*; for discrete features, we split by each category.

Stopping Criteria:

- All labels in the node are identical.
- Number of samples $<$ min_samples_split.
- Maximum depth reached: max_depth $= 5$.
- Gain for best split $<$ min_gain $= 0.01$.

Evaluation on Test Set (80/20 split):

$$\text{Accuracy} = 0.9400, \quad \text{Precision} = 0.9492, \quad \text{Recall} = 0.9492$$

10.6 (f) Overfitting Analysis

Tree Size on Clean Data:

- Max depth: 6
- Nodes: 25

Observation: Controlled growth due to pruning constraints.

Noise Simulation: Flip 10% of training labels and retrain:

$$\text{Accuracy}_{\text{noisy}} = 0.8500, \quad \text{Precision} = 0.9231, \quad \text{Recall} = 0.8136$$

Noisy tree size without pruning: depth ≈ 14, nodes ≈ 202 (overfitting).

10.7 (g) Pruning Methods

Tree Type	Max Depth	Nodes	Notes
Clean Data Full Tree	6	25	Controlled growth, high generalizati
Noisy Data Full Tree	14	202	Overfits noise, memorizes mislabele
Noisy Data Pruned Tree	5 (max_depth)	Reduced	Pruning prevents overfitting

Table 3: Summary of Tree Behaviors under Clean vs. Noisy Data and Pruning

```
Post-Pruned Tree Accuracy: 0.9600
Post-Pruned Tree Precision: 0.9375
Post-Pruned Tree Recall: 1.0000
```

Figure 12: Post Pruning

Implemented pruning strategies:

- **Min Gain Threshold:** only split if IG $>$ min_gain.
- **Max Depth:** limit tree growth to depth $\leq$ max_depth.
- **Reduced Error Pruning (optional):** remove subtrees if validation/test accuracy does not decrease.

Effectiveness: These strategies prevent overfitting on noisy datasets while maintaining high accuracy and precision.

10.8 (h) Summary

- Day_ID has high IG but is useless for generalization $\rightarrow$ dropped.

- Temperature threshold 30.05 and Outlook are most informative features.
- Controlled ID3 tree achieves 94% test accuracy, high Precision/Recall.
- Noise simulation confirms pruning constraints reduce overfitting:
  - Full noisy tree (no constraints): depth 14, nodes 202.
  - Pruned noisy tree: depth ≤ 5, nodes reduced, metrics remain high.
- Demonstrates the importance of pruning (max depth, min gain, optional REP) to balance model complexity and generalization.